同濟大學

TONGJI UNIVERSITY

操作系统课程设计

任课教师 方钰

专　　业 计算机科学与技术

学　　号 1952100

姓　　名 胡昊育

2022 年 06 月 03 日

目 录

第一部分 需求分析.....3
1.1 实验内容.....3
1.2 程序任务.....3
1.2.1 程序输入.....3
1.2.2 程序输出.....4
1.2.3 程序功能.....5
1.3 任务分解.....6
1.3.1 基本任务.....6
1.3.2 进阶任务.....6
第二部分 概要设计.....7
2.1 数据结构.....7
2.1.1 内外存索引节点.....7
2.1.2 目录项及文件打开结构.....7
2.1.3 存储资源管理块.....8
2.1.4 用户与用户组.....8
2.1.5 缓存及缓存管理结构.....8
2.1.6 外界磁盘设备.....9
2.1.7 文件系统结构.....9
2.1.8 用户交互结构.....10
2.2 磁盘结构.....10
2.3 模块关系.....11
2.3.1 全部模块调用关系.....11
2.3.2 内外存索引节点的转化.....11
2.3.3 目录和文件与索引节点.....11
2.3.4 缓存与外界磁盘设备.....12
2.3.5 文件系统与其他模块.....12
2.3.6 用户界面与文件系统.....12
2.4 完整程序流程.....12

第三部分 详细设计.....13
3.1 缓存管理结构.....13
3.1.1 申请缓存块.....13
3.1.2 读缓存.....13
3.1.3 写缓存.....14
3.2 外界磁盘结构.....14
3.2.1 读磁盘到缓存.....14
3.2.2 写缓存到磁盘.....15
3.3 文件系统结构.....15
3.3.1 索引节点的分配与释放.....15
3.3.2 索引节点的读入与保存.....16
3.3.3 磁盘块的分配与释放.....17
3.3.4 由文件名打开索引节点.....18
3.3.5 由索引节点读数据.....18
3.3.6 由索引节点写数据.....19
3.3.7 由索引节点删除文件.....20
3.3.8 由索引节点删除目录(及文件).....20
3.4 用户与用户组管理.....20
3.4.1 用户(组)管理的组织形式.....20
3.4.2 用户(组)的添加.....21
3.4.3 用户(组)的删除.....21
3.5 用户界面.....22
3.5.1 用户登录与退出.....22
3.5.2 类 UNIX 文件句柄.....22
第四部分 结果分析.....23
4.1 程序使用说明.....23
4.2 程序运行结果分析.....24
4.3 测试命令及结果分析.....27
第五部分 参考代码.....29

第一部分 需求分析

1.1 实验内容

本次课程设计需要实现一个类 Unix 的二级文件系统。该文件系统使用一个普通的大文件模拟文件卷，我们需要在该文件中构造类 Unix 的磁盘文件结构，并实现常见的文件操作指令，为用户提供必要的文件操作接口和简洁的 UI 界面。此外，我们还需要在实现该文件系统时选择性地实现高速缓存或多用户管理。

1.2 程序任务

该类 Unix 二级文件系统程序需要完成文件管理、缓存管理以及用户管理的基本任务。对于程序的使用者，程序应该隐藏系统内部文件处理的具体逻辑，只留出最直观、最简单的操作接口供用户使用。

为了方便用户的测试和使用，程序应该支持 test、beta 两种指令集，其中 test 指令集是便于进行测试的基础指令集，beta 指令集是便于用户体验的进阶指令集，两种指令集应该使用同一个磁盘文件。在程序开始运行时，程序会让用户选择接下来要使用的指令集，以完成对应的任务。

1.2.1 程序输入

开始运行时，用户需要在命令行输入选择的指令集名称。选择完成后，程序不断读取用户在命令行输入的文件操作指令，并依据指令完成对应的操作。具体支持的输入指令及其参数格式如下：

指令	参数 1	参数 2	参数 3	指令集
cls	-	-	-	test/beta
exit	-	-	-	test/beta
cd	<路径名>	-	-	test/beta
ftest	-	-	-	test
fhelp	-	-	-	test
fformat	-	-	-	test
flist	-	-	-	test
dcreat	<目录名>	<权限 OTC>	-	test
ddelete	<目录名>	-	-	test
fcreat	<文件名>	<权限 OTC>	-	test
fdelete	<文件名>	-	-	test
fshow	<文件名>	-	-	test
fopen	<文件名>	<权限 OTC>	-	test

指令	参数 1	参数 2	参数 3	指令集
fclose	<句柄 fd>	-	-	test
fread	<句柄 fd>	<缓存名>	<读取大小>	test
fwrite	<句柄 fd>	<init>	<写入大小>	test
fwrite	<句柄 fd>	<缓存名>	<写入大小>	test
flseek	<句柄 fd>	<偏移量>	<偏移模式>	test
fcseek	<句柄 fd>	-	-	test
fsize	<句柄 fd>	-	-	test
help	-	-	-	beta
disk	-	-	-	beta
format	-	-	-	beta
ls	-	-	-	beta
path	-	-	-	beta
mkdir	<目录名>	-	-	beta
rmdir	<目录名>	-	-	beta
touch	<文件名>	-	-	beta
remove	<文件名>	-	-	beta
chmod	<路径名>	<权限 OTC>	-	beta
login	<用户名>	-	-	beta
logout	-	-	-	beta
addu	<用户名>	<用户组名>	-	beta
delu	<用户名>	-	-	beta
addg	<用户组名>	-	-	beta
delg	<用户组名>	-	-	beta
show	<文件名>	-	-	beta
cp	<源文件>	<目的文件>	-	beta
win2fs	<win 路径>	<fs 路径>	-	beta
fs2win	<fs 路径>	<win 路径>	-	beta

1.2.2 程序输出

在一般情况下，程序需要在命令行输出当前路径和当前用户名，以便用户根据当前状态进行操作。在特殊情况下，程序需要在命令行输出的内容如下：

① 刚进入系统时，输出操作指引和注意事项；

② 格式化系统时，输出格式化的时间和完成进度；

③ 指令正常执行时，输出一定的成功反馈；

④ 指令或指令参数出错时，输出对应的错误提示；

⑤ 请求帮助或查看路径时，输出对应指令集的帮助和当前完整路径；

⑥ 查看文件时，输出文件对应的二进制下的 ASCII 码对应字符；

1.2.3 程序功能

根据实验题目的要求，程序需要实现的基本功能有：

① 对一个大文件进行读写，并将该文件视为磁盘来管理；

② 创建基本的目录结构，并拷贝指定文件到文件系统中；

③ 提供指令接口，帮助用户管理磁盘，以及磁盘中的目录和文件；

④ 读取并识别用户输入的指令，根据指令完成正确的操作；

⑤ 及时对用户的操作给予反馈，根据不同输入返回不同结果；

⑥ 可以通过命令行新建、打开、写入、读取文件，可以定位文件指针；

除以上基本功能外，程序还要支持对用户和用户组的管理，对用户权限进行区分，并实现高速缓存的管理。程序支持的各个指令的功能定义如下：

指令	指令集	功能
cls	test/beta	清空命令行
exit	test/beta	退出文件系统
cd	test/beta	进入指定文件夹
ftest	test	一步执行指定的测试指令
fhelp	test	查看 test 指令集的说明
fformat	test	格式化文件系统
flist	test	显示当前目录下的文件信息
dcreat	test	新建文件夹
ddelete	test	删除文件夹及其子文件
fcreat	test	新建文件(返回文件句柄)
fdelete	test	删除文件
fshow	test	显示文件
fopen	test	打开文件(返回文件句柄)
fclose	test	关闭文件
fread	test	读取文件内容到 abc
fwrite	test	初始化文件内容
fwrite	test	将 abc 中内容写入文件
flseek	test	修改文件指针
fcseek	test	返回文件指针
fsize	test	返回 fd 文件大小
help	beta	查看 beta 指令集的说明
disk	beta	显示磁盘使用情况
format	beta	格式化文件系统
ls	beta	显示当前文件夹列表
path	beta	显示当前路径
mkdir	beta	新建文件夹
rmdir	beta	删除文件夹

指令	指令集	功能
touch	beta	新建文件
remove	beta	删除文件
chmod	beta	修改文件(夹)权限
login	beta	用户登录
logout	beta	用户退出
addu	beta	添加用户
delu	beta	删除用户
addg	beta	添加用户组
delg	beta	删除用户组
show	beta	显示文件
cp	beta	文件系统内文件复制
win2fs	beta	将 Windows 文件内容复制到 FS 文件系统文件
fs2win	beta	将 FS 文件系统文件内容复制到 Windows 文件

1.3 任务分解

1.3.1 基本任务

根据要求，本次实验需要完成的基本任务按顺序有：

① 构建并实现文件系统的磁盘结构，包括 Superblock、Inode 等；

② 构建并实现文件的目录结构和文件的打开结构；

③ 构建并实现高速缓存和磁盘结构，实现文件系统的底层读写；

④ 搭建文件系统的 UI 界面，实现必须的文件操作接口；

⑤ 按要求编写主程序，格式化文件系统结束后，等待用户输入；

⑥ 模拟用户测试过程，测试程序使之通过要求的测试；

1.3.2 进阶任务

除以上基本任务外，时间充裕的情况下，还可以选择完成：

① 在 UI 结构和文件管理结构中添加用户和用户组管理；

② 包装基本指令，去除句柄等底层操作手段，简化指令参数；

③ 完善 UI 交互和命令行效果，添加更多丰富的指令；

以上就是本次实验需要完成的任务。除编写代码外，还需要撰写相关的程序文档和实验报告，这里不再赘述。

第二部分 概要设计

2.1 数据结构

2.1.1 内外存索引节点

Inode 和 DiskInode 是文件的内外存索引结构，它们存储着一个文件的基本信息。根据系统所需要实现的功能，Inode 类和 DiskInode 类除了构造函数和析构函数外，包含的相同数据变量有：

①（4 字节）mode：文件工作方式；

②（4 字节）nlink：文件勾连数；

③（2 字节）uid：所属 User 的 id；

④（2 字节）gid：所属 Group 的 id；

⑤（4 字节）size：文件大小(字节)；

⑥（4 字节）addr[10]：文件逻辑块号和物理块号转换索引表；

⑦（4 字节）atime：最后访问时间；

⑧（4 字节）mtime：最后修改时间；

除相同的数据变量外，内存索引节点还有一个 4 字节的 i_number 变量，表示该节点对应的外存索引节点在 Inode 区的编号，方便之后更新。

此外，为了文件管理结构直接调用 Inode 相关的读写函数，在程序设计时，直接将 Inode 需要使用的函数写在了 FileSystem 类下，在 Inode 中只保留基本的数据变量，以简化与 DiskInode 类的相互转化。

2.1.2 目录项及文件打开结构

DirectoryEntry 是目录项结构，它存储着该目录中的文件名(28 字节)以及该文件对应的外存索引节点号(4 字节)。

File 是文件打开结构，它包含的数据成员有：

①（2 字节）f_uid：打开文件的用户 id；

②（2 字节）f_gid：打开文件的用户组 id；

③（Inode 指针）f_inode：指向文件对应的内存索引节点；

④（4 字节）f_offset：当前的文件读写指针位置；

这两类除了构造函数和析构函数外，没有其他的成员函数。

2.1.3 存储资源管理块

SuperBlock 是文件系统的存储资源管理块结构，占有 1024 个字节，管理着磁盘的文件数据区和 Inode 区。该类除了构造函数和析构函数外，不包含其他成员函数，其包含的数据成员有：

①（4 字节）s_inum：Inode 总个数；

②（4 字节）s_ifnum：空闲 Inode 总个数；

③（4 字节）s_ninode：直接管理的空闲外存 Inode 数；

④（4 字节）s_inode[100]：直接管理的空闲外存 Inode 索引表；

⑤（4 字节）s_bnum：磁盘盘块总数；

⑥（4 字节）s_bfnum：空闲盘块总数；

⑦（4 字节）s_nfree：直接管理的空闲盘块数；

⑧（4 字节）s_free[100]：直接管理的空闲盘块索引表；

⑨（4 字节）padding[50]：填充 SuperBlock 至 1024 字节；

2.1.4 用户与用户组

为了实现多用户、多用户组管理的功能，在设计系统时借鉴了某 GitHub 项目的方式(见参考文献)，没有设计单独的用户类，而是使用加密文件的形式记录所有的用户和用户组信息。

每个用户包含的数据成员有：用户名(28 字节)、密码(256 字节)、用户 id(2 字节)，以及用户组 id(2 字节)；每个用户组包含的数据成员有：用户组名(28 字节)、用户组 id(2 字节)，以及该用户组下所有用户的 id(2 字节)。这些信息都按照一定顺序存在加密文件 etc/user 和 etc/group 当中，所有用户都没有权限直接访问这两个文件，只能由管理员用户通过指令修改。

2.1.5 缓存及缓存管理结构

Buf 是单个缓存控制块类，该类没有任何的成员函数，只有两组 Buf 指针，以及 3 个数据成员，具体如下：

①（Buf 指针）b_forw 和 b_back：设备队列；

②（Buf 指针）av_forw 和 av_back：自由队列、I/O 请求队列；

③（char 指针）b_addr：指向该缓存控制块管理的缓冲区首地址；

④（4 字节）b_blkno：读写的磁盘逻辑块号；

⑤（4 字节）b_flags：缓存控制块标志位(B_NONE/B_DONE/B_DELWRI)；

而 BufferManager 是缓存管理类，它主要实现了缓存块的组织和管理，以对外接 IO 设备实现高速读写，除了构造函数和析构函数外，该类还有与缓存块的获取、分配、释放、读写的相关的成员函数(之后介绍)，以及以下数据成员：

①（Buf）bFreeList：自由队列控制块；

②（DiskDevice 指针）disk：磁盘设备的指针；

③（Buf）m_Buf[NBUF]：缓存控制块数组；

④（1 字节）Buffer[NBUF][BUFFER_SIZE]：缓存区数组；

2.1.6 外界磁盘设备

DiskDevice 是外界磁盘设备类。因为该文件系统只有磁盘这一个 IO 设备，依次将 Unix 的设备管理类简化为磁盘设备类。该类除了构造函数和析构函数外，还有与磁盘读写、链接缓存管理的其他成员函数，其包含的数据成员有：

①（BufferManager 指针）bm：缓存管理结构指针；

②（1 字节）disk_name[NAME_SIZE]：磁盘文件名；

③（Buf）bDiskList：设备队列、I/O 队列控制块；

2.1.7 文件系统结构

FileSystem 是文件系统的核心类。它整合了磁盘的 SuperBlock 结构、系统的缓存管理结构，以及外界磁盘设备结构。该类除了构造函数和析构函数外，还实现了与内存索引节点相关的管理函数、文件存储块相关的管理函数、文件目录项相关的管理函数，以及文件操作接口的底层函数。

除众多成员函数外，FileSystem 包含的数据成员有：

①（SuperBlock）spb：存储资源管理对象；

②（DiskDevice）disk：磁盘设备对象；

③（BufferManager）buffer：缓存管理对象；

2.1.8 用户交互结构

UserInterface 是文件系统的用户交互界面类，它包装了 FileSystem 结构，让文件系统的操作接口更高级。该类除了构造函数和析构函数外，还实现了 test 指令集、beta 指令集的各个操作接口函数，以及文件系统运行的流程函数。

此外，该类还借助部分成员变量实现了用户管理、登陆管理、打开文件管理，以及打开文件的打开权限管理。这些数据成员包括：

① (FileSystem) fs：文件系统；

② (字符串) local_host：主机名；

③ (4 字节) cur_uid：用户 id；

④ (4 字节) cur_gid：用户组 id；

⑤ (4 字节) is_login：是否登录；

⑥ (字符串) cur_user：用户名；

⑦ (字符串) cur_group：用户组名；

⑧ (字符串) cur_dir：当前路径；

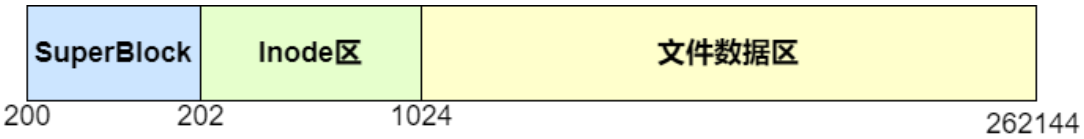
⑨ (4 字节) cur_file_cnt：当前文件数；

⑩ (File 指针) cur_flist[FLIST_SIZE]：当前打开文件表；

⑪ (4 字节) cur_file_mode[FLIST_SIZE]：当前打开文件权限表；

2.2 磁盘结构

整个类 Unix 文件系统的磁盘的内部结构如下：

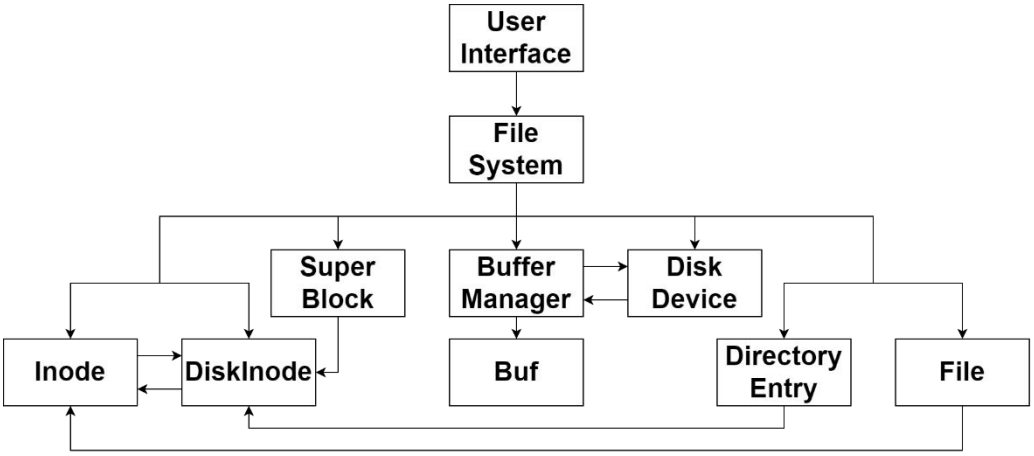


磁盘总大小为 128 MB，共 262144 个逻辑块，每个逻辑块的大小为 512 字节。其中，SuperBlock 占用两个逻辑块，共 1024 字节；Inode 区占用 822 个逻辑块；剩下的文件数据区占有 261120 个逻辑块，单个文件最多占有 33030 个逻辑块。

2.3 模块关系

2.3.1 全部模块调用关系

以上各个模块的调用关系和链接关系大致如下：(箭头指向被调用的模块)



2.3.2 内外存索引节点的转化

Inode 和 DiskInode 两个结构拥有几乎完全相同的数据结构。其中 DiskInode 对应的是静态的文件信息，Inode 对应的是打开的文件信息。为了便于二者转换，在这个项目中，我们使用 memcpy_s()函数直接拷贝这个两个类的实体的前 64 个字节的数据，这样可以快速的完成文件基本信息的载入和保存。

由于使用内存拷贝的方式完成 Inode 节点的载入和保存，因此 DiskInode 和 Inode 类的数据成员的前 64 个字节的数据排列的顺序必须完全一致，否则会出现数据对应出错的情况，影响程序运行。

2.3.3 目录和文件与索引节点

目录项中的 m_ino 数据成员，表示的是该文件对应的外存索引节点号，根据这一成员，我们可以将目录项和对应的文件的 DiskInode 对应起来。而文件打开结构中的 f_inode 是该文件对应内存 Inode 的指针，根据这一成员，我们可以将打开的文件和它的 Inode 对应起来。

目录项存贮在目录文件当中，而目录文件也通过相应的索引节点来索引。在一个逻辑块中，最多可以存储 16 个目录项，一个目录文件也可以占用多个盘块。

2.3.4 缓存与外界磁盘设备

由于该文件系统只有磁盘这一个 IO 设备，因此省略设备管理器，直接连接缓存管理器和外界磁盘。由于本次的课程设计中不需要维护 NODEV 队列，因此缓存管理中的 bFreeList 是自由队列控制块，只使用到了 av_back 和 av_forw。

而外界磁盘设备的 bDiskList 是它的设备队列控制块和 I/O 请求队列控制块，这里直接使用 Buf 类，减少工作量。

缓存管理类和外界磁盘设备类需要在初始化时设置双方的联系，因此在设计时，在两个类中都预留了一个指向对方的指针，并编写了特定的初始化函数。在这两个变量声明后，通过调用初始化函数，就可以连接缓存管理与磁盘设备。

2.3.5 文件系统与其他模块

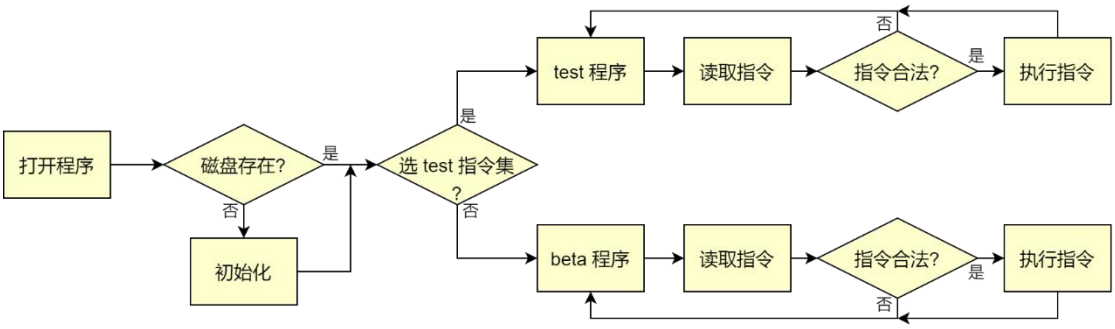
文件系统结构整合统一了其他结构。除了声明了存储资源管理、磁盘设备、缓存管理三大结构的实体，它还实现了 Inode 管理、磁盘块读写、文件及目录管理的相关函数，和每个模块都关系密切。

2.3.6 用户界面与文件系统

用户界面结构包装了 FileSystem 结构，预留出了更符合用户习惯的操作接口，同时借助部分成员变量实现了用户管理、登陆管理、打开文件管理，以及打开文件的打开权限管理等 FileSystem 无法单独实现的功能。

2.4 完整程序流程

程序的总体流程如下：

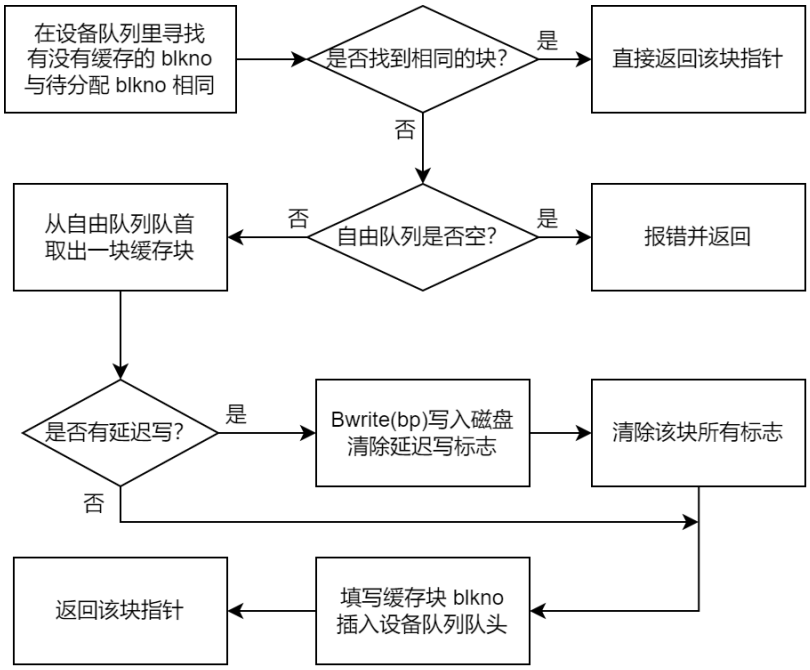


第三部分 详细设计

3.1 缓存管理结构

3.1.1 申请缓存块

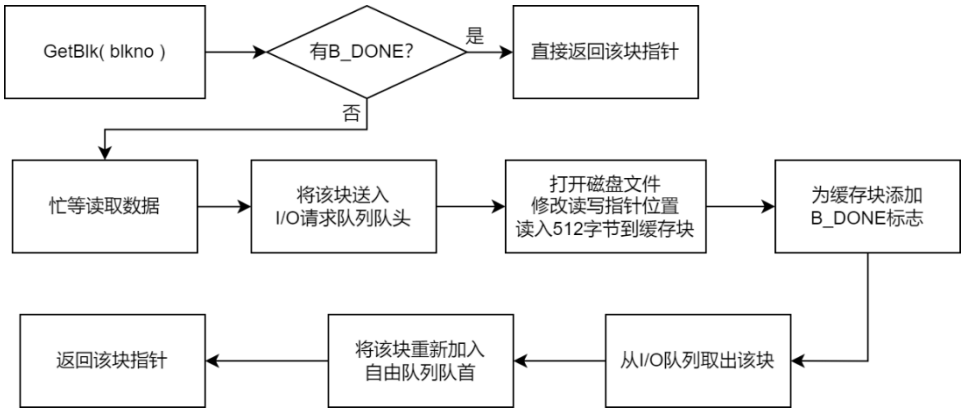
申请缓存块函数 GetBlk(int blkno)的返回值是一个缓存块指针，它的参数 blkno 是我们需要分配缓存的逻辑块的块号。该函数的主要流程如下：



其中，判断自由队列是否为空后，由于设备 I/O 队列采用忙等的方式，因此自由队列只会在内核初始化出错时时才会为空。因此，该函数会在自由队列空时报错，否则继续下一步。

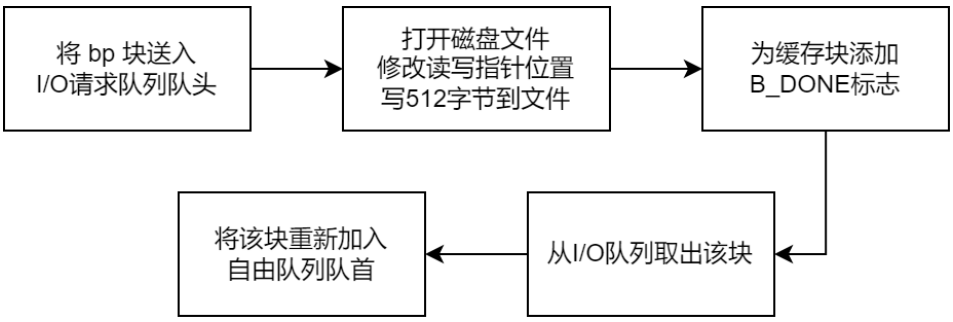
3.1.2 读缓存

读缓存块函数 Bread(int blkno)的返回值是一个缓存块指针，它的参数 blkno 是我们需要读入缓存的逻辑块的块号。该函数的主要流程如下：



3.1.3 写缓存

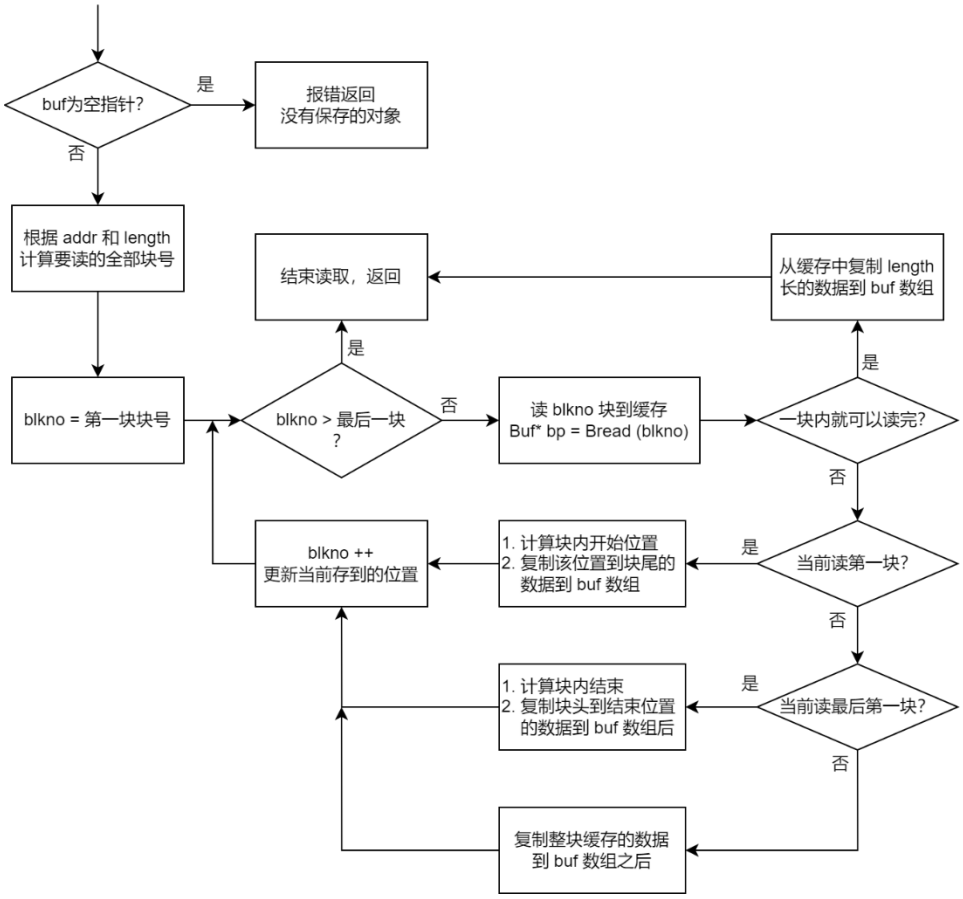
写缓存块函数 Bread(Buf* bp)的参数 bp 是我们要写的缓存块指针。该指针指向的缓存已经有了我们需要的全部信息。该函数的主要流程如下：



3.2 外界磁盘结构

3.2.1 读磁盘到缓存

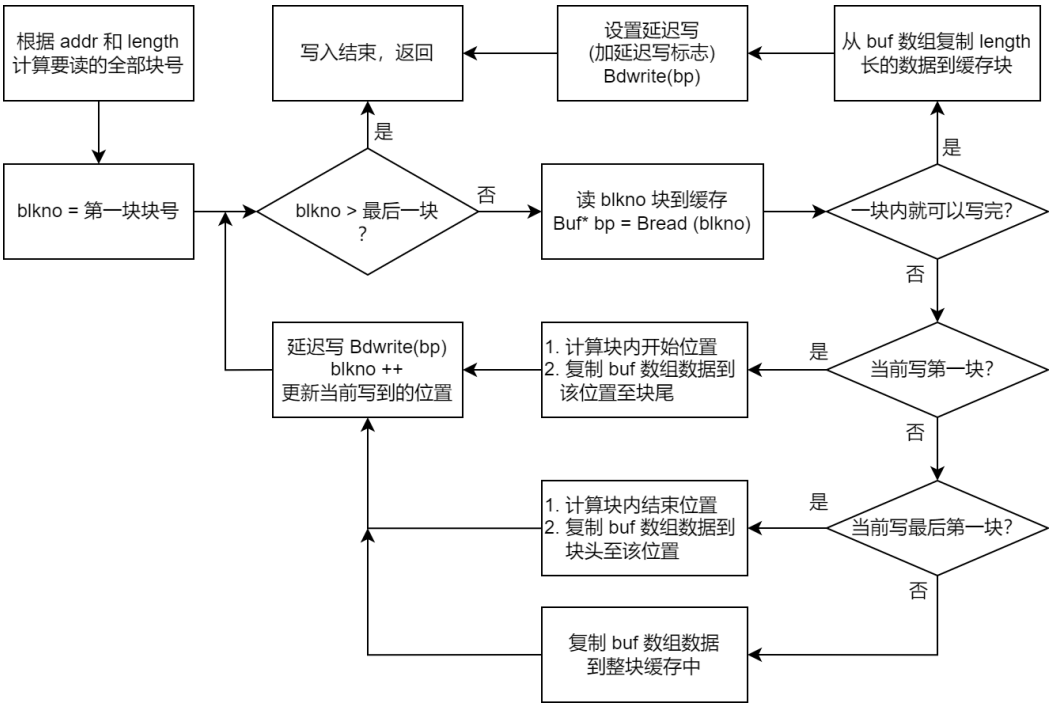
读磁盘信息到缓存函数 Dread(char* buf, int addr, int length)的参数中，buf 是我们存储读出信息的字符数组头指针，addr 是我们要读取磁盘的起始地址，length 是我们要读出的数据长度(多少字节)。该函数的主要流程如下：



该函数调用了 Bread(blkno)函数，使用 memcpy_s()完成数据的拷贝。

3.2.2 写缓存到磁盘

写磁盘信息到缓存函数 Dwrite(char* buf, int addr, int length)的参数中，buf 是我们存储代写信息的字符数组头指针，addr 是我们要写入的磁盘的起始地址，length 是我们要写入的数据长度(多少字节)。该函数的主要流程如下：



在 Dwrite 当中，并没有直接将缓存块的数据写入磁盘，而是为缓存块设置了延迟写标志 B_DELWRI，这样在 Bread()中再次将该缓存块分配给别的逻辑块时，才会将数据全部写入磁盘中。

3.3 文件系统结构

3.3.1 索引节点的分配与释放

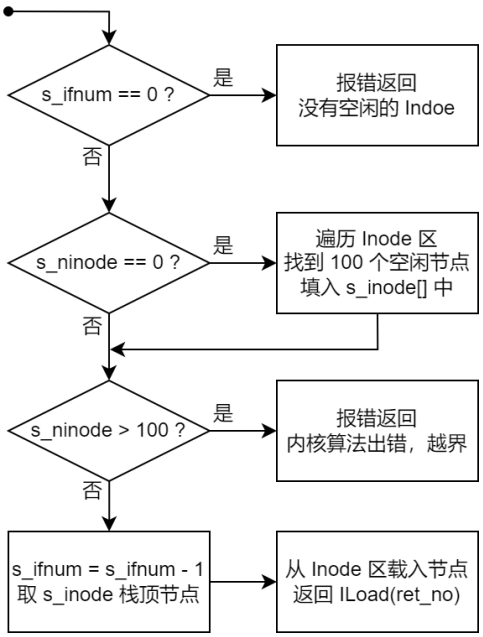
索引节点的分配函数 IAlloc()的返回值是一个 Inode 索引节点。这一分配过程与 SuperBlock 对 Inode 区管理的变量有关：

① Spb.s_ifnum：Inode 区剩余的空闲 Inode 个数；

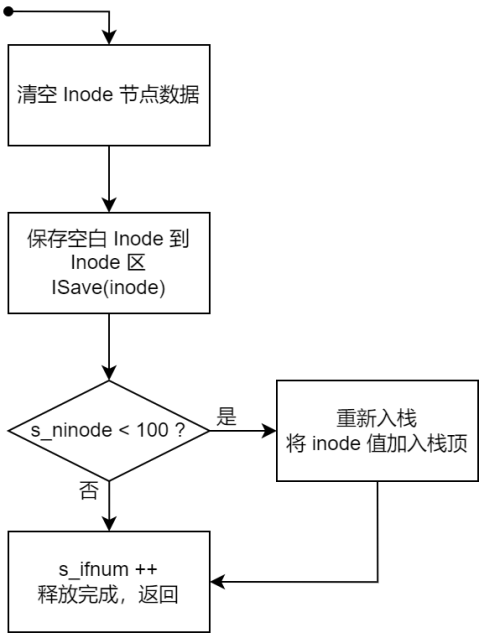
② Spb.s_ninode：当前直接管理的 Inode 个数(最大 100 个)；

③ Spb.s_inode：当前直接管理的 Inode 列表，是它们在 Inode 区的编号；

文件系统对直接管理的 Inode 以栈的形式管理。由于 Indoe 数量有限，释放 Inode 节点的函数 IFree(Inode inode)的逻辑比较简单，当手头的 100 空闲 Inode 用完后，直接在 Inode 区重新搜索 100 就好。两者的算法流程如下：



IAlloc()



IFree(Inode inode)

3.3.2 索引节点的读入与保存

索引节点的读入函数 ILoad(int i_no)的返回值是一个 Inode 索引节点，它的参数 i_no 是待读入 Inode 在 Inode 区的编号。该函数的流程如下：

① 创建一个 DiskInode 对象 d_inode、一个 Inode 对象 inode；

② 计算该 Indoe 的起始地址，调用 Dread()读入 d_inode；

③ 使用 memcpy_s()复制 d_inode 到 inode 前 64 字节；

④ 记录 inode 节点的 i_number 为 i_no；

⑤ 返回 inode 对象；

索引节点的保存函数 ISave(Inode inode)的参数 inode 是待存入 Inode 区的 inode 对象。该函数的流程如下：

① 创建一个 DiskInode 对象 d_inode；

② 使用 memcpy_s()复制 inode 前 64 字节到 d_inode；

③ 计算该 Indoe 的起始地址，调用 Dwrite()写 d_inode 到磁盘；

这两个函数中包含了 DiskInode 和 Inode 节点的转化。关于二者关系的说明可以查看第二部分中 2.3.2 的说明。

3.3.3 磁盘块的分配与释放

磁盘块的分配函数 BAlloc()的返回值是一个分配到的盘块的编号 ret_no；磁盘块的释放函数 BFree(int no)的参数 no 是待释放盘块的编号。这两个过程都与 SuperBlock 对文件存储块的管理变量有关：

① Spb.s_bfnum：文件数据区剩余的空闲盘块个数；

② Spb.s_nfree：当前直接管理的空闲盘块个数(最大 100 个)；

③ Spb.s_free：当前直接管理的空闲盘块列表，是在文件数据区的编号；

文件系统对直接管理的空闲盘块以栈的形式管理。由于磁盘块数量较多，所以空闲盘块每 100 个为一组，链接记录在文件数据区中，只有最后一组空闲盘块由 SuperBlock 管理。每一组记录存储着如下 n+1 个数据(n<=100)：

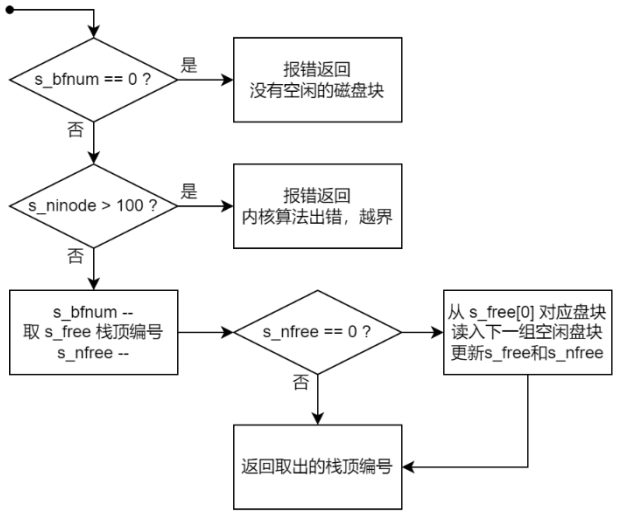
① 第 1 个数据：该组空闲盘块的总个数 n；

② 第 2 个数据：位于栈底的空闲盘块的编号，同时链接上一组记录的块；

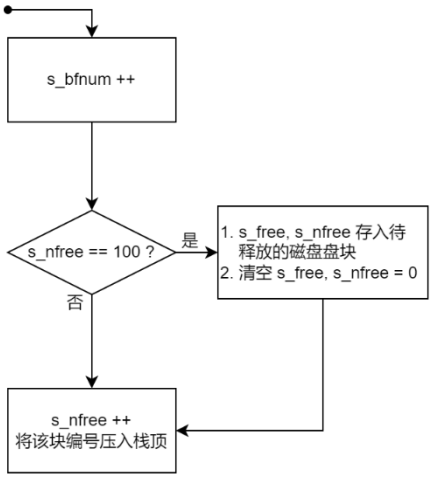
③ 第 3~(n+1)个数据：其余的该组的空闲盘块号，大的一头是栈顶；

当 Spb 直接管理的空闲盘块超过 100 个时，我们会将这组记录写入当前回收盘的前 101 * 4 个字节，同时更新 Spb 中有关的 3 个变量；当 Spb 直接管理的空闲盘块只剩下 0 个时，我们会读取 s_free[0]对应盘块的前 101 * 4 个字节，更新 Spb 中和空闲盘块有关的这 3 个变量。

明确空闲盘块记录的链接关系后，两者的算法流程如下：



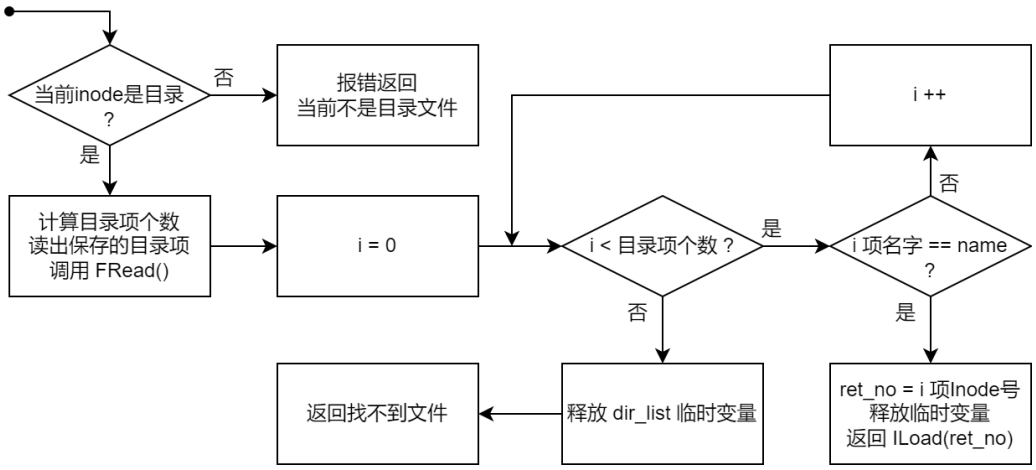
BAlloc()



BFree(int no)

3.3.4 由文件名打开索引节点

根据路径打开 Inode 的函数 FFind(Inode& dir_inode, char* name)的返回值是文件(目录)名为 name 的 Inode 节点。它的参数 dir_inode 是当前待搜索的目录文件的 Inode 节点地址，name 是该目录文件下待查找的文件(目录)名。该函数的整体流程如下：



在已知文件完整路径的情况下，我们可以按照以下步骤从根目录开始，找到并打开文件对应的 Inode 节点：

① 按照“/”字符分解完整路径，形成路径向量 route；

② 令 Inode 型变量 cur_dir 为根目录；

③ 在遍历 route 的同时，令 cur_dir = FFind(cur_dir, route[i])；

④ route 向量遍历完后，cur_dir 就是要找的文件(目录)的 Inode 节点；

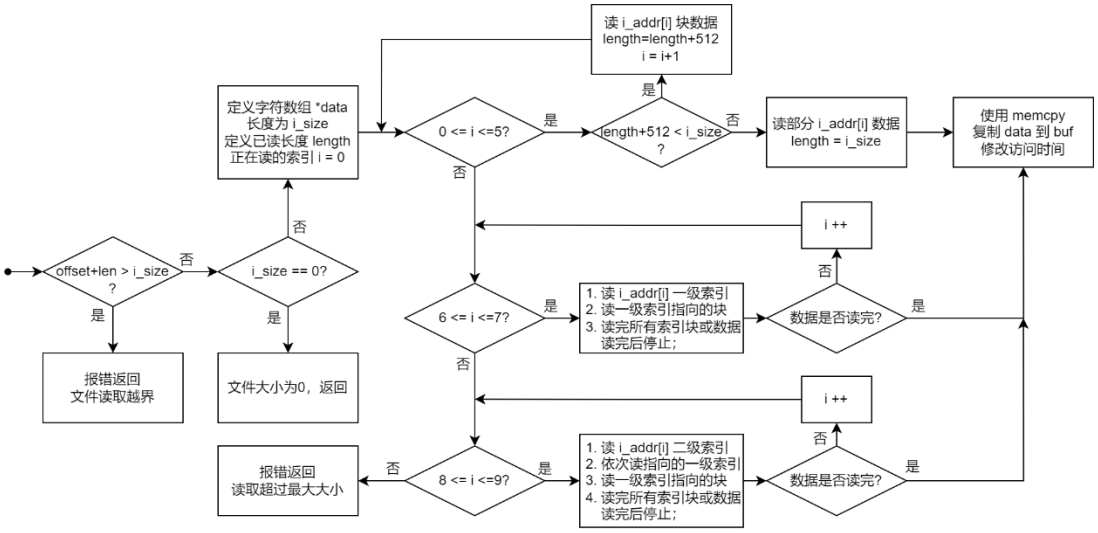
若我们要从当前打开的某个目录打开文件的 Indoe 节点，我们可以直接令一开始声明的 Inode 变量 cur_dir 当前路径的 Inode 节点，之后再遍历即可。

3.3.5 由索引节点读数据

根据内存索引节点 Inode 读取文件的指定位置、指定长度的数据到字符数组的函数是 FRead(Inode& cur_inode, char* buf, int offset, int len)。

它有四个参数，其中 cur_inode 是待读取文件对应的 Inode 节点，buf 是存取数据的字符数组，offset 是从文件头起的读取开始位置的偏移量，len 是要读取的数据长度。

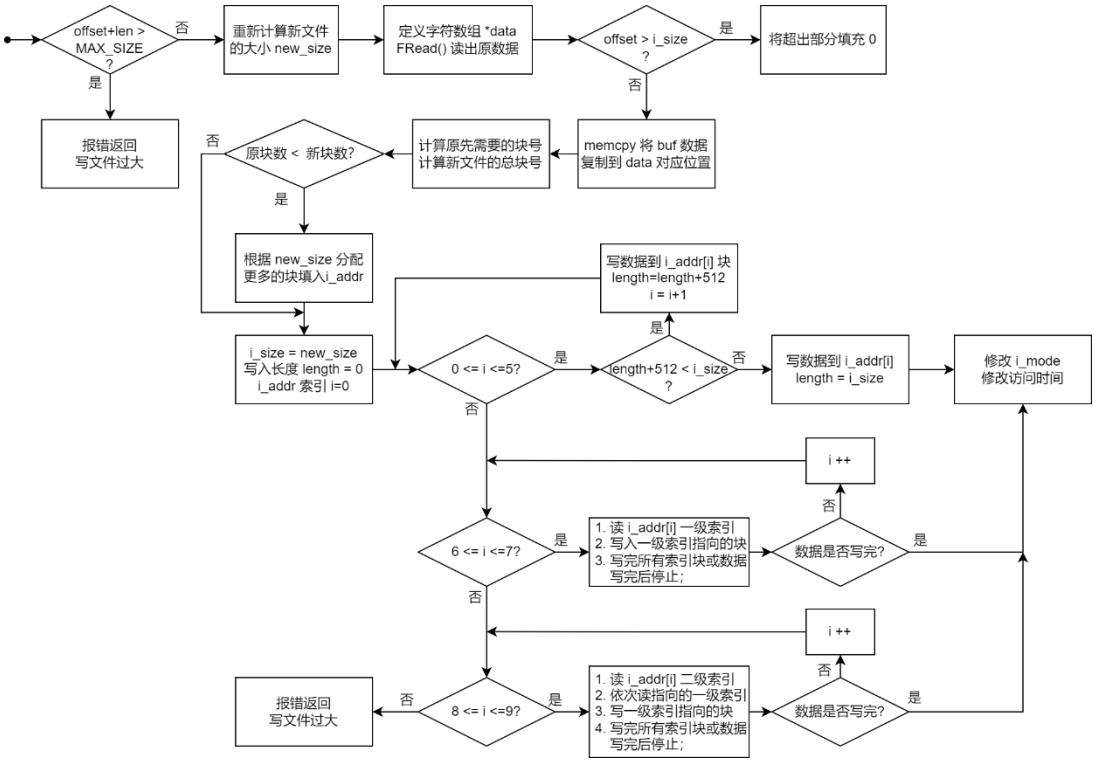
该函数的调用了 Dread()函数，会依据 inode 节点中 i_addr 的值，依次读取直接索引、一级间接索引、二级间接索引的数据到 buf 中。具体流程如下：



3.3.6 由索引节点写数据

根据内存索引节点 Inode 向文件在指定位置写入指定长度的字符数组的数据的函数是 FWrite(Inode& cur_inode, char* buf, int offset, int len)。

它有四个参数，其中 cur_inode 是待写入文件对应的 Inode 节点，buf 是待存入数据的字符数组，offset 是待写入位置对文件头的偏移量，len 是要写入的数据的长度。它会依据要写入数据的大小，首先填充 i_addr 中直接索引、一级间接索引、二级间接索引的数据块的编号(使用 BAlloc()分配)，之后再按照分配的数据块，依次写入要写入的数据。具体流程如下：



3.3.7 由索引节点删除文件

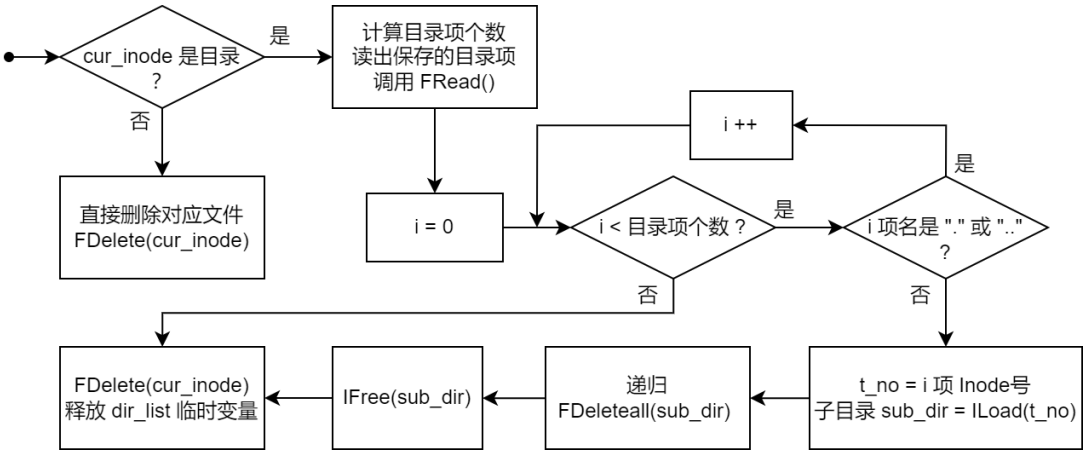
由索引节点 Inode 删除磁盘对应的文件的函数 FDelete(Inode& cur_inode)的参数 cur_inode 是待删除文件的内存索引节点。函数的逻辑是根据文件的大小以及索引节点的 i_addr 记录的块号,对每个物理块逐个调用 BFree(blkno)函数。

在释放完文件所有的数据磁盘数据之后，函数会修改 inode 节点对应的信息，包括修改 i_size 为 0，修改 i_mode，以及修改访问时间等。

由于函数的逻辑与 FRead 类似，只不过删去了一些判断大小的过程，这里不再绘制它的算法流程图。

3.3.8 由索引节点删除目录(及文件)

由索引节点删除目录项及其对应子项的函数 FDeleteall(Inode& cur_inode)的参数 cur_inode 是待删除目录文件的内存索引节点。其主要流程如下：



3.4 用户与用户组管理

3.4.1 用户(组)管理的组织形式

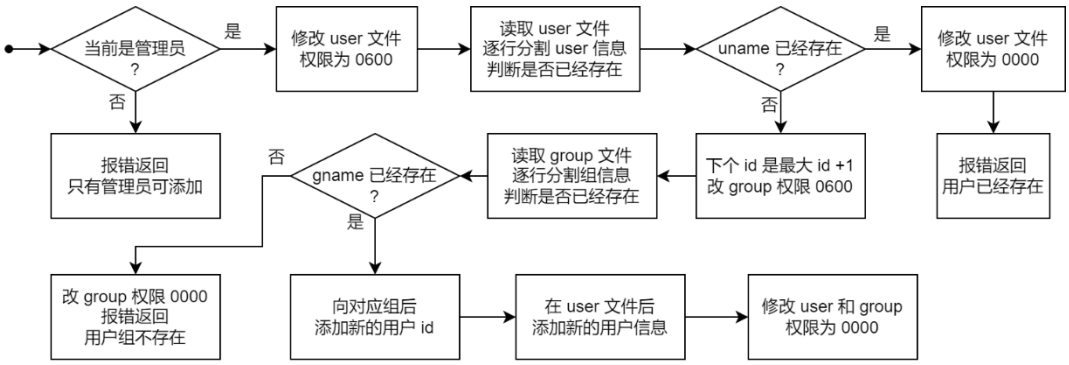
为了实现多用户、多用户组管理的功能，系统使用加密文件的形式记录所有的用户和用户组信息。所有有效的用户和用户组的信息均存放在加密文件 etc/user 和 etc/group 当中，所有用户都没有权限直接访问这两个文件，只有在管理员通过 addu、delu、addg、delg 指令修改用户和用户组信息时，才会在这几个函数内短暂的开放文件的读写权限。

用户信息包括：用户名(28 字节)、密码(256 字节)、用户 id(2 字节)，以及用户组 id(2 字节)；用户组信息包括：用户组名(28 字节)、用户组 id(2 字节)，

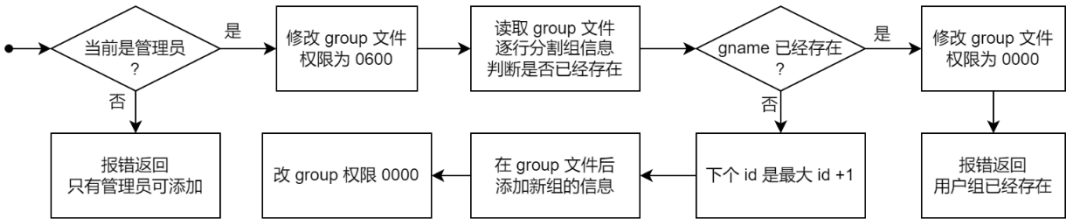
以及该用户组下所有用户的 id(2 字节)。这些信息都按照一定顺序储存在文件中，在需要访问时短暂开放。默认的管理员组 id 为 0，名称为 root；默认的管理员 id 为 0，用户组 id 为 0，用户名是 admin，密码是 password。

3.4.2 用户(组)的添加

添加用户的函数是 addu(string uname, string gname, string passwd)，其参数中 uname 和 gname 分别是用户名和用户组名，passwd 是密码。添加用户组的函数 addg(string gname)的参数 gname 是用户组名。它们的逻辑如下：



addu()



addg()

3.4.3 用户(组)的删除

删除用户的函数是 delu(string uname)，其参数中 uname 是用户名；删除用户组的函数 delg(string gname)的参数 gname 是用户组名。它们的逻辑与添加的逻辑基本相同，都是先判断是否有权限修改，再开放权限，然后判断用户名和用户组名的存在性，之后在做相应的修改。这里不再绘制它们的具体流程图，但又以下 3 点值得注意：

① 管理员用户和管理员组不可以删除；

② 在删除用户组时，对应的组内的 user 也要全部删除；

③ 在删除用户时，对应用户组内也要删除对应的 user 的 id；

3.5 用户界面

3.5.1 用户登录与退出

用户登录的函数 login(string name, string passwd)的返回值是一个布尔变量，成功登陆为 1，不成功为 0。它的参数 name 是用户输入的账户名，passwd 是用户输入的密码。它的主要流程如下：

① 修改 user 文件权限 0400，读取所有 user 信息；

② 将 user 信息按行分割，依次判断各用户的名字和密码是否与参数相同；

③ 若全部遍历后没有相同的，修改 user 权限 0000，返回 0；若有相同的，则记录对应的 uid，并执行下一步；

④ 修改 group 文件权限 0400，读取所有 group 信息；

⑤ 查找该用户在哪一个用户组下，记录对应的用户组 gid；

⑥ 修改 group 和 user 文件的权限为 0000；

⑦ 设置当前的目录为根目录，返回 1；

用户退出的逻辑比较简单，只需要设置登录状态为 false，当前用户 id 和用户组 id 为 -1，并设置当前目录为根目录即可，不再赘述。

3.5.2 类 UNIX 文件句柄

为了模拟类 Unix 的文件操作句柄，该系统在 test 指令集中，设置了使用句柄操作文件的函数。在用户界面结构中，变量 cur_flist[10]记录着当前打开文件的指针，每个指针在数组中的位置，就是该文件的句柄。变量 cur_file_mode[10]存储的是对应文件的打开模式，它和 cur_flist[10]一一对应。

用户在打开一个文件后，会自动插入到打开文件列表当中，并记录这次打开文件的权限，从而获得一个文件的操作句柄。用户可以依据句柄完成 fclose、fread、fwrite、flseek、fcseek、fsize 等操作。

第四部分 结果分析

4.1 程序使用说明

(注意：程序需要 128 MB 的空间，请确保.exe 所在磁盘下有足够空间)

① 在第一次运行程序时，程序会自动创建磁盘文件，并会在控制台有提示：

```
-NOTICE: 正在加载文件系统, 开始时间: 03:06:40(不应超过2分钟)
```

② 待磁盘初始化完成后，我们看到如下的过程信息以及结束提示：

```
-NOTICE: 正在加载文件系统, 开始时间: 03:06:40(不应超过2分钟)
 - text size:        198 B => wrote size:      198 B
 - report size:   488431 B => wrote size:   488431 B
 - photo size:    161361 B => wrote size:   161361 B
-NOTICE: 文件系统加载成功, 结束时间: 03:07:03
```

③ 之后，程序会出现 4 条整体的使用提示，请用户仔细阅读：

```
-------------------------------------------------------------------------------/
| @Project :  FileSystem
| @Author  :  Ray
| @GitHub  :  https://github.com/RayCorleone/FileSystem
| @Notice  :
|     1.退出系统务必使用 exit 指令, 否则有一定概率出错;
|     2.系统出错后, 执行 format/fformat 指令可以格式化系统;
|     3.管理员名: admin, 管理员密码: password, 初始用户组: root;
|     4.系统有 test 和 beta 两种指令集, 一旦选择后, 在本次运行过程中不可更改;
|         (1)test指令集: 使用文件句柄操作, 管理员自动登录;
|         (2)beta指令集: 需要自行登录管理员账户, 使用更高级的指令结构;
-------------------------------------------------------------------------------/
```

④ 阅读完提示后，请选择本次测试使用那种指令集，可输入 test 或 beta。

其中 test 指令集使用的是 fd 句柄，beta 指令集没有句柄的操作：

```
-请选择指令集(结束请输入exit)[test/beta]: test
```

⑤ 进入 test 指令集后，可以输入 fhelp 获取指令集的帮助：

```
-NOTICE: 当前文件系统使用test指令集, 输入 fhelp 获取帮助.
admin@RAYBEAST:root# fhelp
- cls                                    - 清空命令行
- ftest                                  - 一步实现测试
- fhelp                                  - 显示命令清单
- fformat                                - 格式化文件系统
- flist                                  - 显示当前文件夹列表
- cd      <dir name>                     - 进入文件夹
- dcreat  <dir name> <mode(OTC)>         - 新建文件夹
- ddelete <dir name>                     - 删除文件夹
- fcreat  <file name> <mode(OTC)>        - 新建文件(返回文件句柄)
- fdelete <file name>                    - 删除文件
- fshow   <file name>                    - 显示文件
- fopen   <file name> <mode(OTC)>        - 打开文件(返回文件句柄)
- fclose  <句柄 fd>                      - 关闭文件
- fread   <句柄 fd> <buf abc> <size>     - 读取文件内容到abc
- fwrite  <句柄 fd> <init> <size>        - 初始化文件内容
- fwrite  <句柄 fd> <buf abc> <size>     - 将abc中内容写入文件
- flseek  <句柄 fd> <offset> <ptrname>   - 修改文件指针
- fcseek  <句柄 fd>                      - 返回文件指针
- fsize   <句柄 fd>                      - 返回fd文件大小
- exit                                   - 退出系统
admin@RAYBEAST:root#
```

⑥ 进入 beta 指令集后，可输入 help 获取指令集的帮助：

```
-NOTICE: 当前文件系统使用beta指令集, 输入 help 获取帮助.
@RAYBEAST:root$ help
- help                    - 显示命令清单    (未登录可用)
- cls                     - 清空命令行      (未登录可用)
- format                  - 格式化文件系统  (未登录可用)
- login   <user name>     - 用户登录        (未登录可用)
- exit                    - 退出系统        (未登录可用)
                ......登录查看更多指令......
p.s.管理员名:admin | 密码:password | 初始用户组:root

@RAYBEAST:root$
```

⑦ beta 指令集下，输入 login admin，之后再输入 password 后，可登录管理员账户，解锁更多指令；

⑧ 建议在退出系统时，使用 exit 指令，这样有利于系统结果的保存；

⑨ 若在使用过程中出现错误，请使用格式化指令 fformat(test 指令集下)或 format(beta 指令集下)重新初始化系统；

⑩ 有其他问题，欢迎联系：1406547351@qq.com；

4.2 程序运行结果分析

由于 test 指令集将在 4.3 完成大部分的展示，这里只展示 beta 指令集。

我们在未登录或登录其他用户时，当前的命令行字符为$(左)；当我们使用 login 登录管理员账户后，命令行字符变为#(右)。

```
@RAYBEAST:root$
```

```
admin@RAYBEAST:root#
```

登陆后我们在更目录下新建一个 try 文件夹，然后通过 cd 进入文件夹后，新建一个 a.txt 文件，最后列出这一目录下的情况。这一过程的指令和结果是：

(1)mkdir try　(2)cd try　(3)touch a.txt　(4)ls

```
admin@RAYBEAST:root# mkdir try
admin@RAYBEAST:root# cd try
admin@RAYBEAST:root/try# touch a.txt
admin@RAYBEAST:root/try# ls
类型&读写权限    链接数    文件(目录)名          文件主   用户组   大小       修改时间
d|rwx|rw-|r--    1         .                     admin    root     96 B       2022.6.5 03:37:19
d|rwx|rwx|rwx    6         ..                    admin    root     192 B      2022.6.5 03:37:09
-|rw-|rw-|r--    1         a.txt                 admin    root     0 B        2022.6.5 03:37:19
admin@RAYBEAST:root/try#
```

我们之后将 window 系统中的已经写好的 a.txt 复制该磁盘的 a.txt 中，然后展示磁盘中 a.txt 的内容。这一过程的指令和结果是：

(1)win2fs D:/a.txt ./a.txt　(2)show a.txt

```
admin@RAYBEAST:root/try# win2fs D:/a.txt a.txt
admin@RAYBEAST:root/try# show a.txt
This is a test file for win2fs !!!!
admin@RAYBEAST:root/try#
```

接下来，我们 cd 到 home 文件夹，在 home 文件夹下新建一个 b.txt 文件，把 a.txt 的文件内容拷贝过来，再展示 b.txt 内容。这一过程的指令和结果是：

(1)cd ../home　　(2)touch b.txt

(3)cp ../try/a.txt b.txt　　(4)show b.txt

```
admin@RAYBEAST:root/try# cd ../home
admin@RAYBEAST:root/home# touch b.txt
admin@RAYBEAST:root/home# cp ../try/a.txt b.txt
admin@RAYBEAST:root/home# show b.txt
This is a test file for win2fs !!!!
admin@RAYBEAST:root/home#
```

现在我们修改 b.txt 的权限为文件主不可读不可写，同组用户可读可写，不同组用户可读不可写，然后我们再展示 b.txt。这一过程的指令和结果是：

(1)chmod b.txt 064　　(2)show b.txt

```
admin@RAYBEAST:root/home# chmod b.txt 064
admin@RAYBEAST:root/home# show b.txt
-!ERROR: 权限不足，缺少读权限.
admin@RAYBEAST:root/home#
```

我们先修改 home 文件夹的权限为 777，再建立一个新的用户组 tjer，并为它添加一个用户 ray，密码是 pass；之后我们为 root 组添加一个 try 的新用户，密码是 pass；然后我们退出管理员账户，登录 ray 账户后查看 b.txt。这一过程的指令和结果是：

(1)chmod . 777　　(2)addg tjer　　(3)addu ray tjer (pass)

(4)addu try root (pass)　　(5)logout

(6)login ray (pass)　　(7)cd home　　(8)show b.txt

```
admin@RAYBEAST:root/home# chmod . 777
admin@RAYBEAST:root/home# addg tjer
admin@RAYBEAST:root/home# addu ray tjer
password: ****
admin@RAYBEAST:root/home# addu try root
password: ****
admin@RAYBEAST:root/home# logout
@RAYBEAST:root$ login ray
password: ****
ray@RAYBEAST:root$ cd home
ray@RAYBEAST:root/home$ show b.txt
This is a test file for win2fs !!!!
ray@RAYBEAST:root/home$
```

我们再尝试删除 b.txt，这一过程的指令和结果是：

(1)remove b.txt

```
ray@RAYBEAST:root/home$ remove b.txt
-!ERROR: 权限不足，缺少写权限.
ray@RAYBEAST:root/home$
```

现在我们登录 try 账号，再次尝试删除 b.txt，这一过程的指令和结果是：

(1)logout　　(2)login try (pass)

(3)cd home　　(4)remove b.txt

```
admin@RAYBEAST:root# logout
@RAYBEAST:root$ login try
password: ****
try@RAYBEAST:root$ cd home
try@RAYBEAST:root/home$ remove b.txt
try@RAYBEAST:root/home$ ls
```

现在重新登录管理员账户，删除 try 文件夹和 try 用户，再删除 tjer 用户组，再 cd 到 home/reports 目录，查看当前的完整路径，并查看磁盘使用空间。这一过程的指令和结果是：

(1)logout　　(2)login admin (password)　　(3)rmdir try

(4)delu try　　(5)delg tjer　　(6)cd ./home/reports

(7)path　　(8)disk

```
try@RAYBEAST:root/home$ logout
@RAYBEAST:root$ login admin
password: ********
admin@RAYBEAST:root# rmdir try
admin@RAYBEAST:root# delu try
admin@RAYBEAST:root# delg tjer
admin@RAYBEAST:root# cd ./home/reports
admin@RAYBEAST:root/home/reports# path
[root]/home/reports
admin@RAYBEAST:root/home/reports# disk
文件系统        |总空间        |已使用        |未使用        |空间比        |Inode数        |空闲Inode

FileSystem.img  |133693440     |2321408       |131372032     |1.74          |6576           |6525

admin@RAYBEAST:root/home/reports#
```

最后尝试登录 ray 用户，这一过程的指令和结果是：

(1)logout　　(2)login ray (pass)

```
admin@RAYBEAST:root/home/reports# logout
@RAYBEAST:root$ login ray
password: ****
-!ERROR: 登录失败.
@RAYBEAST:root$
```

使用 exit 指令退出程序即可。

注意：

① 以上未展示 cls、exit 和 fs2win 的效果，用户可以自行测试；

② 指令情况过多，不再完全展示。用户可以自行尝试，测试更多情况；

③ 系统中可能存在 Bug，如遇到，欢迎联系：1406547351@qq.com；

4.3 测试命令及结果分析

在选择 test 指令集后，测试命令及其解释如下：

① dcreat test 777　　# 创建 test 文件夹(权限 777)

② cd test　　# cd 进入 test 文件夹

③ fcreat Jerry 664　　# 创建 Jerry 文件(权限 664)，返回句柄为 0

④ fwrite 0 init 800　　# 向句柄为 0 文件写入 800 的字节

⑤ flseek 0 500 0　　# 将句柄为 0 文件的读写指针移到 500 字节处

⑥ fread 0 abc 500　　# 从句柄为 0 文件读出 500 字节到 abc 字符串

⑦ fwrite 0 abc 500　　# 向句柄为 0 文件写入 500 字节(写 abc 字符串)

在依次执行以上几步后，我们获得的结果如下：

```
-NOTICE: 当前文件系统使用test指令集, 输入 fhelp 获取帮助.
admin@RAYBEAST:root# dcreat test 777
admin@RAYBEAST:root# cd test
admin@RAYBEAST:root/test# fcreat Jerry 664
-NOTICE: 返回值为0
admin@RAYBEAST:root/test# fwrite 0 init 800
-NOTICE: 返回值为800
admin@RAYBEAST:root/test# flseek 0 500 0
-NOTICE: 返回值为500
admin@RAYBEAST:root/test# fread 0 abc 500
-NOTICE: 返回值为300
admin@RAYBEAST:root/test# fwrite 0 abc 500
-NOTICE: 返回值为500
admin@RAYBEAST:root/test#
```

其中，指令③返回值是创建文件时自动生成的文件句柄，这时创建并打开的的文件只有 Jerry 一个，所以分配到的文件句柄为 0。

指令④返回值是成功写入的字节数，init 字符串有 1000 个字符，可以写入 Jerry。这里执行指令④后，成功给 Jerry 文件写入了 800 字节。此时 Jerry 的读写指针应该位于 800 字节处。

指令⑤返回值是当前读写指针的位置，这里使用 0 号方法，将指针定位到了 Jerry 文件的 500 字节处。

指令⑥返回值是实际读取的字节数，由于 Jerry 文件只有 800 字节，当我们从文件的读写指针开始向后读取时，读至末尾就停止了，所以只读了 300 字节。

指令⑦返回值时实际写入的字节数。fwrite 在 abc 数据读完后，会填充空白数据，直至写完 500 个字节。

我们此时再查看指针位置以及 Jerry 中的数据，可以获得如下结果：

```
admin@RAYBEAST:root/test# fcseek 0
-NOTICE: 返回值为1300
admin@RAYBEAST:root/test# fshow 0
-!ERROR: 找不到该文件.
admin@RAYBEAST:root/test# fclose 0
admin@RAYBEAST:root/test# fshow Jerry
abcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklm
nopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyz
abcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklm
nopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyz
abcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklm
nopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZ
ABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLM
NOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZ
ABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTGHIJKLMNOPQRSTUVWXY
ZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKL
MNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXY
ZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKL
MNOPQRST
```

可见此时读写指针到了 800 + 500 = 1300 的位置，Jerry 中的数据也如下图展示，有 1100 字节的完整数据，和空白数据 200 字节。

若用户不想手动输入测试，可以使用 ftest 指令，一步执行所有的测试指令。ftest 的输出结果如下：

```
admin@RAYBEAST:root/test# ftest
-NOTICE: 正在格式化文件系统, 开始时间: 05:42:53(不应超过2分钟).
 - text size:        198 B => wrote size:      198 B
 - report size:   488431 B => wrote size:   488431 B
 - photo size:    161361 B => wrote size:   161361 B
-NOTICE: 文件系统格式化成功, 结束时间: 05:43:54.

(1)新建文件/test/Jerry:
-创建的文件的句柄:0
类型&读写权限    链接数    文件(目录)名                    文件主   用户组   大小        修改时间
d|rwx|rwx|rwx    1         .                               admin    root     96 B        2022.6.5 05:43:54
d|rwx|rwx|rwx    6         ..                              admin    root     192 B       2022.6.5 05:43:54
-|rw-|---|---    1         Jerry                           admin    root     0 B         2022.6.5 05:43:54

(2)向test/Jerry文件写入800字节:
-向句柄0写入800字节
-句柄0的大小:800

(3)将文件指针定位到500字节:
-句柄0修改之前的指针:800
-句柄0修改之后的指针:500

(4)读出500字节到abc:
-从句柄0读出字节数:300
-字符串abc的值:
GHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGH
IJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJ
KLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRST

(5)将abc写回文件:
-向句柄0写入500字节
-句柄0的大小:1300

-写回文件后Jerry的大小:
类型&读写权限    链接数    文件(目录)名                    文件主   用户组   大小        修改时间
d|rwx|rwx|rwx    1         .                               admin    root     96 B        2022.6.5 05:43:54
d|rwx|rwx|rwx    6         ..                              admin    root     192 B       2022.6.5 05:43:54
-|rw-|---|---    1         Jerry                           admin    root     1300 B      2022.6.5 05:43:54

-写回文件后Jerry的内容:
abcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzab
cdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcd
efghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdef
ghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefgh
ijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyzabcdefGHIJKLMNOPQRSTUVWXYZABCDEFGHIJ
KLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKL
MNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMN
OPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZAB
CDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCD
EFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZABCDEF
GHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRST

admin@RAYBEAST:root/test#
```

第五部分 参考代码

[1] Unix V6++ 源代码;

[2] https://github.com/elliotxx/os_filesystem;

[3] 其他网上资料等;